

Planning and Reinforcement Learning in AI Systems

(PARLAI)

Lesson 8b - Approximate Methods

Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

We have discussed:

- Value-based RL: Monte-Carlo and Temporal Difference

We will discuss:

- Policy-based RL
- Model-based RL

All converge to optimal policy assuming a GLIE exploration strategy.

All methods (implicitly) assume

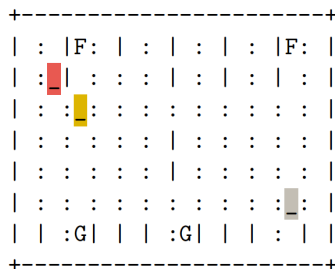
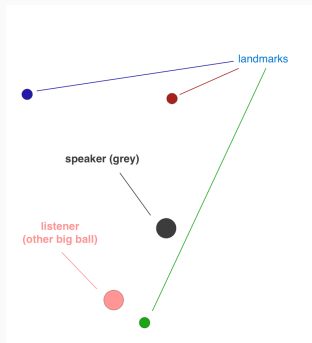
- the world is not too dangerous (no cliffs to fall off during exploration)
- small state spaces

Examples

Linear Function
Approximation

Approximation
for RL

Conclusion



Limitations of methods seen so far ? Ideas ?

- When we represent a value function by a lookup table, every state s has an entry $V(s)$ or every state-action pair s, a has an entry $Q(s, a)$.
- When a problem has a large state space, we can no longer represent V and Q (or the transition and reward functions) as explicit tables.

What to do?

Function Approximation

Linear Function
Approximation

Approximation
for RL

Conclusion

- Never enough training data!
 - Must **generalize** what is learned from one situation to other “similar” new situations
- **Idea:** Instead of using large tables to represent V and Q , use a **parameterized function**
 - The number of parameters should be small compared to number of states (generally exponentially fewer parameters)
- Learn parameters from experience
- When we update the parameters based on observations in one state, then our V and Q estimates will also change for other similar states

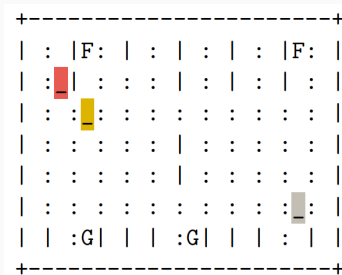
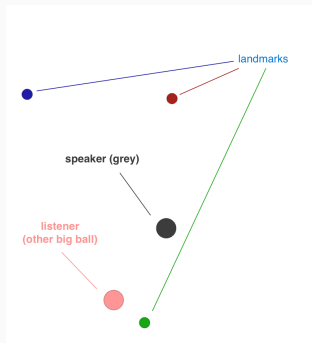
Parameterization facilitates generalization of experience!

Back to the Examples

Linear Function
Approximation

Approximation
for RL

Conclusion



Parameterization ?

- Estimate value function with function approximation

$$\hat{V}(s, \theta) \approx V_{\pi}(s)$$

or

$$\hat{Q}(s, a, \theta) \approx Q_{\pi}(s, a)$$

- Generalise from seen states to unseen states
- Update parameter θ using MC or TD learning.

Types of Value Function Approximation

Linear Function
Approximation

Approximation
for RL

Conclusion

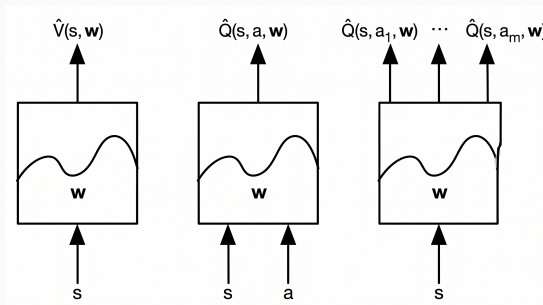


Image by David Silver

Which Function Approximator?

Linear Function
Approximation

Approximation
for RL

Conclusion

Which Function Approximator?

- Linear combinations of features
- Neural network
- Decision tree
- Nearest neighbour
- Fourier / wavelet bases
- ...

Linear Function
Approximation

Approximation
for RL

Conclusion

Which Function Approximator?

- Linear combinations of features
- Neural network
- Decision tree
- Nearest neighbour
- Fourier / wavelet bases
- ...

*Typically, we consider **differentiable function approximators**-
Why?

Furthermore, we require a training method that is suitable for
non-stationary, non-iid data

Linear Function
Approximation

Approximation
for RL

Conclusion

Which Function Approximator?

- Linear combinations of features*
- Neural network*
- Decision tree
- Nearest neighbour
- Fourier / wavelet bases
- ...

*Typically, we consider **differentiable function approximators**-
Why?

Furthermore, we require a training method that is suitable for
non-stationary, non-iid data

Linear Function
Approximation

Approximation
for RL

Conclusion

Linear Function Approximation

Linear Function Approximation

Linear Function
Approximation

Approximation
for RL

Conclusion

A common approximation is to represent $V(s)$ as a weighted sum of the features (linear approximation)

$$V_{\theta}(s) = \theta_0 + \theta_1 f_1(s) + \cdots + \theta_n f_n(s)$$

The approximation accuracy is fundamentally limited by the information provided by the features

Feature Vectors

- Define a set of **state features** $f_1(s), \dots, f_n(s)$
- State represented by a *feature vector*

$$x(s) = \begin{pmatrix} x_1(s) \\ \vdots \\ x_n(s) \end{pmatrix}$$

- For example:
 - Distance of robot from landmarks
 - Trends in the stock market
 - Piece and pawn configurations in chess

Can we always define features that allow for a perfect value function approximation?

Linear Function
Approximation

Approximation
for RL

Conclusion

Feature Vectors

- Define a set of **state features** $f_1(s), \dots, f_n(s)$
- State represented by a *feature vector*

$$x(s) = \begin{pmatrix} x_1(s) \\ \vdots \\ x_n(s) \end{pmatrix}$$

- For example:
 - Distance of robot from landmarks
 - Trends in the stock market
 - Piece and pawn configurations in chess

Can we always define features that allow for a perfect value function approximation?

Yes. but...

Linear Function
Approximation

Approximation
for RL

Conclusion

Table Lookup Features

Linear Function
Approximation

Approximation
for RL

Conclusion

- Assign each state an indicator feature.
- Using table lookup features

$$x(s) = \begin{pmatrix} 1(s = s_1) \\ \vdots \\ 1(s = s_n) \end{pmatrix}$$

- Parameter vector θ gives value of each individual state.

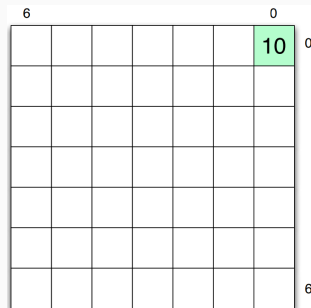
$$\hat{V}(s, \theta) = \begin{pmatrix} 1(s = s_1) \\ \vdots \\ 1(s = s_n) \end{pmatrix} \cdot \begin{pmatrix} \theta_1 \\ \vdots \\ \theta_n \end{pmatrix}$$

limitations? This requires far too many features and gives no generalization.

Example

Grid with no obstacles, deterministic actions Up-Down-Left-Right, no discounting, -1 reward everywhere except +10 at goal.

- Features for $s = (x, y)$:
 $f_1(s) = x, f_2(s) = y$
- Parameterized representation of value function ?



Linear Function
Approximation

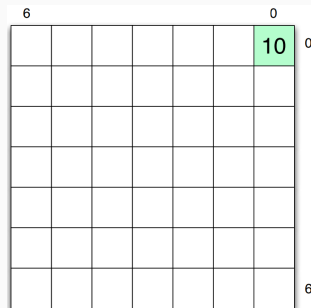
Approximation
for RL

Conclusion

Example

Grid with no obstacles, deterministic actions Up-Down-Left-Right, no discounting, -1 reward everywhere except +10 at goal.

- Features for $s = (x, y)$:
 $f_1(s) = x, f_2(s) = y$
- Parameterized representation of value function ?
 - $V_{\theta}(s) = \theta_0 + \theta_1 x + \theta_2 y$



Linear Function
Approximation

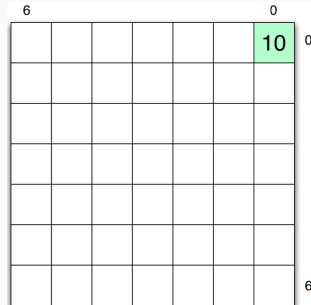
Approximation
for RL

Conclusion

Example

Grid with no obstacles, deterministic actions Up-Down-Left-Right, no discounting, -1 reward everywhere except +10 at goal.

- Features for $s = (x, y)$:
 $f_1(s) = x, f_2(s) = y$
- Parameterized representation of value function ?
 - $V_\theta(s) = \theta_0 + \theta_1 x + \theta_2 y$
- Is there a good linear approximation?



Linear Function
Approximation

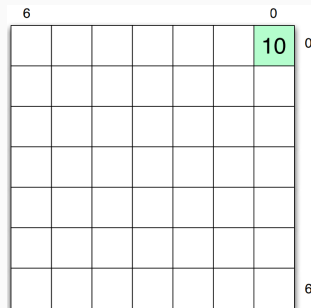
Approximation
for RL

Conclusion

Example

Grid with no obstacles, deterministic actions Up-Down-Left-Right, no discounting, -1 reward everywhere except +10 at goal.

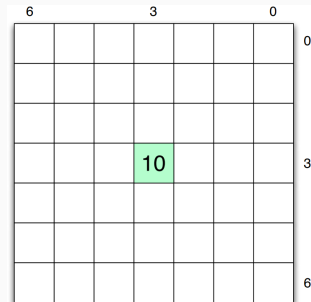
- Features for $s = (x, y)$:
 $f_1(s) = x, f_2(s) = y$
- Parameterized representation of value function ?
 - $V_\theta(s) = \theta_0 + \theta_1 x + \theta_2 y$
- Is there a good linear approximation?
 - Yes.
 - $\theta_0 = 10, \theta_1 = \theta_2 = -1$
 - note: upper right is origin
 - $V_\theta(s) = 10 - x - y$ (subtracts Manhattan distance from goal reward)



What if we change the reward function ?

Grid with no obstacles, deterministic actions Up-Down-Left-Right, no discounting, -1 reward everywhere except +10 at goal.

- Features for $s = (x, y)$:
 $f_1(s) = x, f_2(s) = y$
- $V_\theta(s) = \theta_0 + \theta_1 x + \theta_2 y$
- Is there a good linear approximation?

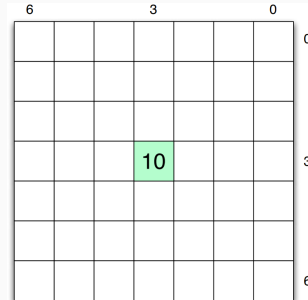


What if we change the reward function ?

Grid with no obstacles, deterministic actions Up-Down-Left-Right, no discounting, -1 reward everywhere except +10 at goal.

- Features for $s = (x, y)$:
 $f_1(s) = x, f_2(s) = y$
- $V_\theta(s) = \theta_0 + \theta_1 x + \theta_2 y$
- Is there a good linear approximation?
- No!

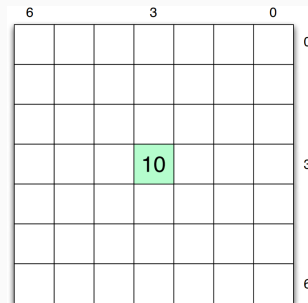
Suggestions?



But What If..

Grid with no obstacles, deterministic actions Up-Down-Left-Right, no discounting, -1 reward everywhere except +10 at goal.

- Features for $s = (x, y)$:
 $f_1(s) = x$, $f_2(s) = y$
 $f_3(s) = |3 - x| + |3 - y|$
- $V_\theta(s) = \theta_0 + \theta_1 x + \theta_2 y + \theta_3 f_3(s)$
- Is there a good linear approximation?
- Yes!
- $\theta_0 = 10, \theta_1 = \theta_2 = 0, \theta_3 = -1$



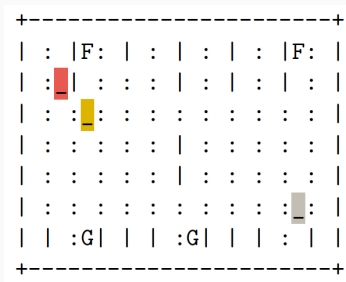
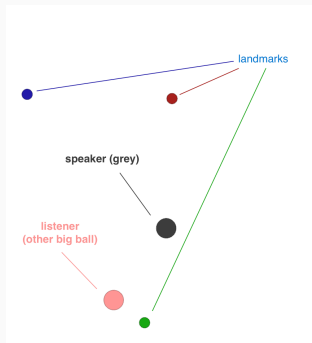
Linear Value Function Approximation

Linear Function
Approximation

Approximation
for RL

Conclusion

What about our domains ?



Linear Value Function Approximation

Linear Function
Approximation

Approximation
for RL

Conclusion

- Define a set of state features $f_1(s), \dots, f_n(s)$
 - The features are used as our representation of states
 - States with similar feature values will be considered to be similar
 - More complex functions require more complex features
$$V_{\theta}(s) = \theta_0 + \theta_1 f_1 + \theta_2 f_2 + \dots + \theta_n f_n(s)$$
- Our goal is to **learn good parameter values** (i.e. feature weights) that approximate the value function well
 - How can we do this?

Linear Value Function Approximation

Linear Function
Approximation

Approximation
for RL

Conclusion

- Define a set of state features $f_1(s), \dots, f_n(s)$
 - The features are used as our representation of states
 - States with similar feature values will be considered to be similar
 - More complex functions require more complex features
$$V_{\theta}(s) = \theta_0 + \theta_1 f_1 + \theta_2 f_2 + \dots + \theta_n f_n(s)$$
- Our goal is to **learn good parameter values** (i.e. feature weights) that approximate the value function well
 - **How can we do this?**
 - Let's try using TD-based RL and somehow update parameters based on each experience.

Approximation for RL

TD-based RL for Linear Approximators

Linear Function
Approximation

Approximation
for RL

Conclusion

TD-based RL for Linear Approximators

- 1 Start with initial **parameter values**
- 2 Execute action from **explore/exploit policy**
- 3 Update estimated model (if model is not available)
- 4 Perform TD update for each parameter: $\theta_i := ?$
- 5 Goto 2

What is a “TD update” for a parameter?

TD-based RL for Linear Approximators

Linear Function
Approximation

Approximation
for RL

Conclusion

TD-based RL for Linear Approximators

- 1 Start with initial parameter values
- 2 Execute action from **explore/exploit policy**
- 3 Update estimated model (if model is not available)
- 4 Perform TD update for each parameter:

$$\theta_i := \theta_i + \alpha(v(s_j) - V_\theta(s_i))f_i(s_j)$$

- 5 Goto 2

What should we use for "target value" $v(s)$?

TD-based RL for Linear Approximators

- 1 Start with initial **parameter values**
- 2 Execute action from **explore/exploit policy**
- 3 Update estimated model (if model is not available)
- 4 Perform TD update for each parameter:

$$\theta_i := \theta_i + \alpha(v(s_j) - V_\theta(s_i))f_i(s_j)$$

- 5 Goto 2

What should we use for "target value" $v(s)$? Use the TD prediction based on the next state s' $v(s) = R(s) + \gamma V_\theta(s')$ the same as previous TD methods, only with approximation.

TD-based RL for Linear Approximators

TD-based RL for Linear Approximators

- 1 Start with initial **parameter values**
- 2 Execute action from **explore/exploit policy**
- 3 Update estimated model (if model is not available)
- 4 Perform TD update for each parameter:

$$\theta_i := \theta_i + \alpha (R(s) + \gamma V_{\theta}(s') - V_{\theta}(s_j)) f_i(s_j)$$

- 5 Goto 2

In what way do we depend on a model ?

Linear Function
Approximation

Approximation
for RL

Conclusion

TD-based RL for Linear Approximators

- 1 Start with initial **parameter values**
- 2 Execute action from **explore/exploit policy**
- 3 Update estimated model (if model is not available)
- 4 Perform TD update for each parameter:

$$\theta_i := \theta_i + \alpha (R(s) + \gamma V_{\theta}(s') - V_{\theta}(s_j)) f_i(s_j)$$

- 5 Goto 2

In what way do we depend on a model ?

- Step 2 requires a model to select greedy action
 - For applications such as Backgammon it is easy to get a simulation-based model
 - For others it is difficult to get a good model
- We can use model-free Q-learning

Q-Learning for Linear Approximators

Features are functions of states and actions:

$$Q_{\theta} = \theta_0 + \theta_1 f_1(s, a) + \dots + \theta_n f_n(s, a)$$

Linear Function
Approximation

Approximation
for RL

Conclusion

Q-Learning for Linear Approximators

- 1 Start with initial parameter values
- 2 Execute action from **explore/exploit policy** giving s' (should converge to greedy policy, i.e., GLIE)
- 3 Perform TD update for each parameter:

$$\theta_i := \theta_i + \alpha \left(R(s) + \gamma \max_{a'} Q_{\theta}(s', a') - Q_{\theta}(s, a) \right) f_i(s, a)$$

- 4 Goto 2

Converges under some conditions.

Reminder: Value-Based and Policy-Based RL

- Value Based
 - Learnt Value Function
 - Implicit policy (e.g. ϵ -greedy)
- Policy Based
 - No Value Function
 - Learnt Policy
- Actor-Critic
 - Learnt Value Function
 - Learnt Policy

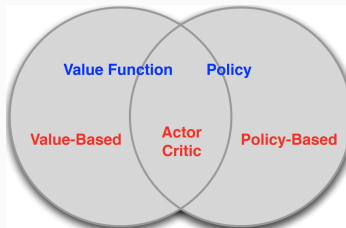


Image by David Silver

Linear Function
Approximation

Approximation
for RL

Conclusion

- So far, we approximated the value or action-value function using parameters θ

$$V_{\theta}(s) \approx V^{\pi}(s)$$

$$Q_{\theta}(s) \approx Q^{\pi}(s)$$

- A policy was generated directly from the value function
 - e.g. using ϵ -greedy
- An alternative is to directly parametrise the policy
$$\pi_{\theta}(s, a) = \mathcal{P}[a|s, \theta]$$
- This is (again) model-free reinforcement learning

Advantages of Policy-Based RL

Linear Function
Approximation

Approximation
for RL

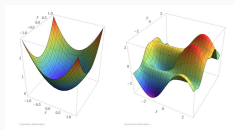
Conclusion

Advantages:

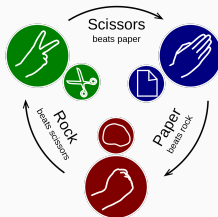
- Better convergence properties
- Effective in high-dimensional or continuous action spaces
- Can learn stochastic policies

Disadvantages:

- Typically converge to a local rather than global optimum
- Evaluating a policy is typically inefficient and high variance



Example: Rock-Paper-Scissors



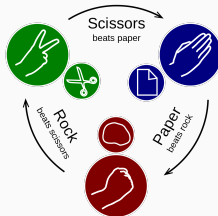
- Two-player game of rock-paper-scissors
 - Scissors beats paper
 - Rock beats scissors
 - Paper beats rock
- Consider policies for iterated rock-paper-scissors

Linear Function
Approximation

Approximation
for RL

Conclusion

Example: Rock-Paper-Scissors



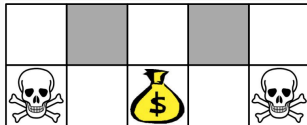
- Two-player game of rock-paper-scissors
 - Scissors beats paper
 - Rock beats scissors
 - Paper beats rock
- Consider policies for iterated rock-paper-scissors
 - A deterministic policy is easily exploited
 - A uniform random policy is optimal (i.e. Nash equilibrium)

Linear Function
Approximation

Approximation
for RL

Conclusion

Example: Aliased Gridworld



- The agent cannot differentiate the grey states
- Consider features of the following form (for all N, E, S, W)

$$\phi(s, a) = 1(\text{wall to } N, a = \text{move } E)$$

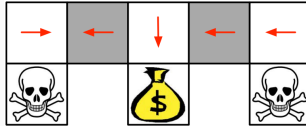
- Compare value-based RL, using an approximate value function

$$Q_{\theta}(s, a) = f(\phi(s, a), \theta)$$

- To policy-based RL, using a parametrised policy

$$\pi_{\theta}(s, a) = g(\phi(s, a), \theta)$$

Example: Aliased Gridworld (continued 1)



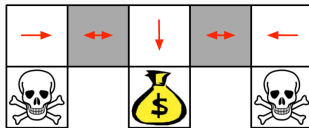
- Under aliasing, an optimal deterministic policy will either
 - move W in both grey states (shown by red arrows)
 - move E in both grey states
- Either way, it can get stuck and never reach the money
- Value-based RL learns a near-deterministic policy
 - e.g. greedy or ϵ -greedy
- So it will traverse the corridor for a long time

Linear Function
Approximation

Approximation
for RL

Conclusion

Example: Aliased Gridworld (continued 2)



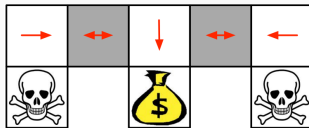
- An optimal **stochastic policy** will randomly move E or W in grey states

$$\pi_{\theta}(\text{wall to N and S}, \text{move E}) = 0.5$$

$$\pi_{\theta}(\text{wall to N and S}, \text{move W}) = 0.5$$

- It will reach the goal state in a few steps with high probability
- Policy-based RL can learn an optimal stochastic policy

Example: Aliased Gridworld (continued 2)



- An optimal stochastic policy will randomly move E or W in grey states

$$\pi_{\theta}(\text{wall to N and S}, \text{move E}) = 0.5$$

$$\pi_{\theta}(\text{wall to N and S}, \text{move W}) = 0.5$$

- It will reach the goal state in a few steps with high probability
- Policy-based RL can learn an optimal stochastic policy

Policy Objective Functions

Linear Function
Approximation

Approximation
for RL

Conclusion

- Goal: given policy $\pi_{\theta}(s, a)$ with parameters θ , find best θ
- But how do we measure the quality of a policy π_{θ} ?

Policy Objective Functions

Linear Function
Approximation

Approximation
for RL

Conclusion

- Goal: given policy $\pi_{\theta}(s, a)$ with parameters θ , find best θ
- But how do we measure the quality of a policy π_{θ} ?
 - In episodic environments we can use the start value
 - In continuing environments we can use the average value or average reward per time-step

- Policy based reinforcement learning is an optimisation problem
- Find θ that maximises $J(\theta)$
- Some approaches do not use gradient
 - Hill climbing
 - Simplex / amoeba / Nelder Mead
 - Genetic algorithms
- Greater efficiency often possible using gradient
 - Gradient descent
 - Conjugate gradient
 - Quasi-newton
- We focus on gradient descent, many extensions possible
- And on methods that exploit sequential structure

Computing Gradients By Finite Differences

Linear Function
Approximation

Approximation
for RL

Conclusion

- To evaluate policy gradient of $\pi_{\theta}(s, a)$
- For each dimension $k \in [1, n]$
 - Estimate kth partial derivative of objective function w.r.t. θ
 - By perturbing θ by small amount ϵ in kth dimension

$$\frac{\partial J(\theta)}{\partial \theta_k} \approx \frac{J(\theta + \epsilon u_k) - J(\theta)}{\epsilon}$$

where u_k is unit vector with 1 in kth component, 0 elsewhere

- Uses n evaluations to compute policy gradient in n dimensions
- Simple, noisy, inefficient - but sometimes effective
- **Works for arbitrary policies, even if policy is not differentiable**

Model-Based with Function Approximators

What will we try to approximate? How?

Linear Function
Approximation

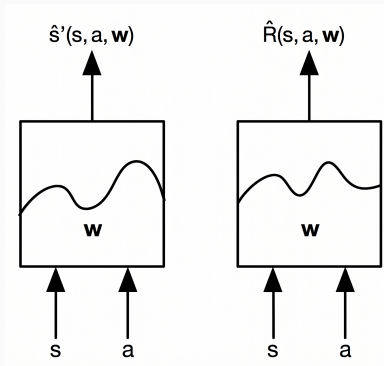
Approximation
for RL

Conclusion

Model-Based with Function Approximators

What will we try to approximate? How?

Reward and transition function.



Linear Function
Approximation

Approximation
for RL

Conclusion

Conclusion

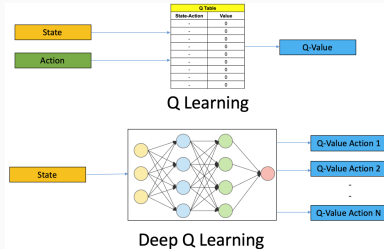
Deep Reinforcement Learning

Linear Function
Approximation

Approximation
for RL

Conclusion

- Combines reinforcement learning (RL) with deep learning to solve complex decision-making problems.



Foundation Models as General Function Approximators

Linear Function
Approximation

Approximation
for RL

Conclusion

A foundation model is a very powerful parameterized function.

Instead of manually designing features, we learn representations from massive amounts of data.

The same architecture can approximate many functions used in RL:

$V_{\theta}(s)$ Value function

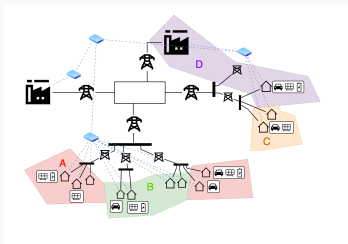
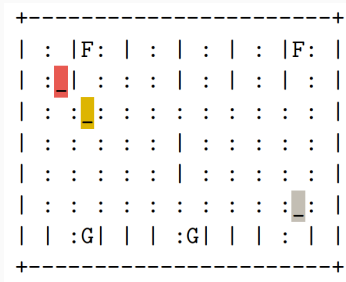
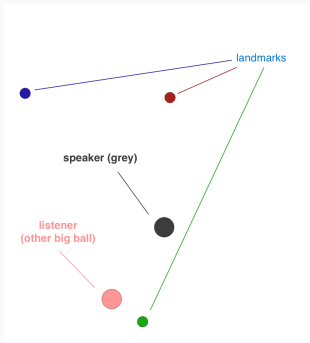
$Q_{\theta}(s, a)$ Action-value function

$\pi_{\theta}(a|s)$ Policy

$P_{\theta}(s'|s, a)$ Dynamics model

$R_{\theta}(s, a)$ Reward model

RL Examples



Linear Function
Approximation

Approximation
for RL

Conclusion